Contents

Arquitectura Hexagonal en NestJS — Guía desde cero **1**
1. La idea, en una frase 1
2. Los 4 conceptos que tienes que memorizar 1
3. Estructura de carpetas recomendada 2
4. Comandos de terminal para crear toda la estructura 2
5. Ejemplo completo, paso a paso 4
6. ¿Qué se hace primero y por qué? 8
7. Cómo queda esto con PostgreSQL específicamente 9
8. El gran beneficio: testear sin NestJS 13
9. Errores comunes de principiante (para que no caigas en ellos) 14
10. Chuleta rápida (resumen mental) 14
11. Siguiente paso natural: Domain-Driven Design (DDD) 14

Arquitectura Hexagonal en NestJS — Guía desde cero

1. La idea, en una frase

Tu lógica de negocio (el "dominio") **no debe saber nada** de NestJS, de la base de datos, de HTTP, ni de ninguna librería externa. Todo lo "externo" (Postgres, Express, Redis, un microservicio, etc.) se conecta al dominio a través de **puertos** (interfaces) y **adaptadores** (implementaciones concretas de esas interfaces).

Piensa en un hexágono: en el centro está tu negocio (las reglas puras, en JavaScript/TypeScript plano). Alrededor, en cada "lado" del hexágono, hay una puerta (puerto) por donde algo entra o sale. Lo que se conecta a esa puerta (una API REST, una base de datos, un cron job) es el adaptador.

¿Por qué te importa esto? - Puedes cambiar de Postgres a MongoDB sin tocar tu lógica de negocio. - Puedes testear tus reglas de negocio con test unitarios normales, sin levantar Nest, sin base de datos, sin mocks complicados. - El código queda ordenado: sabes exactamente dónde va cada cosa.

---

2. Los 4 conceptos que tienes que memorizar

Concepto	Qué es	Ejemplo
Dominio	Las entidades y reglas de negocio puras. Cero imports de Nest, TypeORM, etc.	Clase `Product`, reglas como "el precio no puede ser negativo"
Puerto (Port)	Una interfaz TypeScript que declara "necesito que alguien haga esto por mí"	`interface ProductRepository { save(p: Product): Promise<void> }`
Caso de uso (Use Case / Application Service)	La lógica que orquesta el dominio para cumplir una acción concreta	`CreateProductUseCase`
Adaptador (Adapter)	La implementación real de un puerto, usando tecnología concreta	`ProductRepositoryTypeOrm implements ProductRepository`

Hay dos tipos de puertos/adaptadores, y esto confunde mucho a los que empiezan:

- **Puertos de entrada (driving/primary)**: algo de afuera "empuja" hacia tu dominio. Ejemplo: un `Controller` HTTP llama a tu caso de uso. El adaptador de entrada es el Controller.
- **Puertos de salida (driven/secondary)**: tu dominio "necesita" algo de afuera. Ejemplo: guardar en base de datos. El puerto es la interfaz `ProductRepository`, el adaptador es `ProductRepositoryTypeOrm`.

Regla de oro: **las flechas de dependencia siempre apuntan hacia el dominio**. El dominio no importa nada de infraestructura. La infraestructura importa (implementa) las interfaces del dominio.

---

3. Estructura de carpetas recomendada

Esta es la estructura más usada en proyectos NestJS reales (por módulo/feature):

```
src/
└── products/
    ├── domain/
    │   ├── product.entity.ts          # entidad pura, sin decoradores de Nest
    │   ├── product-repository.port.ts # interfaz (puerto de salida)
    │   └── errors/
    │       └── invalid-price.error.ts
    │
    ├── application/
    │   ├── create-product.use-case.ts
    │   ├── get-product.use-case.ts
    │   └── ports/
    │       └── (a veces los puertos de entrada van aquí)
    │
    ├── infrastructure/
    │   ├── persistence/
    │   │   ├── product.orm-entity.ts     # entidad de TypeORM/Prisma (NO es la del dominio)
    │   │   └── product-typeorm.repository.ts # adaptador que implementa el puerto
    │   └── http/
    │       ├── product.controller.ts         # adaptador de entrada
    │       └── dto/
    │           └── create-product.dto.ts
    │
    └── products.module.ts             # aquí se hace el "cableado" con DI
```

Idea clave: **domain** no depende de nada. **application** depende solo de **domain**. **infrastructure** depende de **domain** y **application** (para implementarlos), nunca al revés.

---

4. Comandos de terminal para crear toda la estructura

Antes de ir al ejemplo paso a paso, aquí tienes los comandos para crear de una vez todas las carpetas y archivos vacíos del módulo `products`, respetando la estructura de la sección 3. Ejecuta esto parado en la raíz de tu proyecto NestJS (donde está la carpeta `src/`):

```bash
# Carpetas
mkdir -p src/products/domain/errors
```

```
mkdir -p src/products/application/ports
mkdir -p src/products/infrastructure/persistence
mkdir -p src/products/infrastructure/http/dto

# Archivos de domain/
touch src/products/domain/product.entity.ts
touch src/products/domain/product-repository.port.ts
touch src/products/domain/errors/invalid-price.error.ts

# Archivos de application/
touch src/products/application/create-product.use-case.ts
touch src/products/application/get-product.use-case.ts

# Archivos de infrastructure/persistence
touch src/products/infrastructure/persistence/product.orm-entity.ts
touch src/products/infrastructure/persistence/product-typeorm.repository.ts

# Archivos de infrastructure/http
touch src/products/infrastructure/http/product.controller.ts
touch src/products/infrastructure/http/dto/create-product.dto.ts

# El módulo, en la raíz del feature
touch src/products/products.module.ts
```

Todo en un solo bloque (copiar y pegar directo):

```
mkdir -p src/products/domain/errors src/products/application/ports src/products/infrastructure/
touch src/products/domain/product.entity.ts \
      src/products/domain/product-repository.port.ts \
      src/products/domain/errors/invalid-price.error.ts \
      src/products/application/create-product.use-case.ts \
      src/products/application/get-product.use-case.ts \
      src/products/infrastructure/persistence/product.orm-entity.ts \
      src/products/infrastructure/persistence/product-typeorm.repository.ts \
      src/products/infrastructure/http/product.controller.ts \
      src/products/infrastructure/http/dto/create-product.dto.ts \
      src/products/products.module.ts
```

Verificar que quedó bien

```
# Muestra el árbol de carpetas creado (si no tienes 'tree', instálalo con: sudo apt install tre
tree src/products
```

Deberías ver exactamente esto:

```
src/products
├── application
│   ├── create-product.use-case.ts
│   ├── get-product.use-case.ts
│   └── ports
├── domain
│   ├── errors
│   │   └── invalid-price.error.ts
│   ├── product-repository.port.ts
```

```
│   └── product.entity.ts
├── infrastructure
│   ├── http
│   │   ├── dto
│   │   │   └── create-product.dto.ts
│   │   └── product.controller.ts
│   └── persistence
│       ├── product-typeorm.repository.ts
│       └── product.orm-entity.ts
└── products.module.ts
```

Nota sobre el CLI de Nest

El comando `nest generate` (`nest g module products`, `nest g controller products`, etc.) **no conoce esta estructura de carpetas** — por defecto genera todo plano dentro de `src/products/`. Puedes usarlo igual y luego mover los archivos a mano a su carpeta correspondiente (`domain/`, `application/`, `infrastructure/`), o simplemente crear los archivos con `touch` como arriba y escribir el contenido tú mismo. La mayoría de equipos que usan hexagonal en NestJS terminan haciendo esto último, porque el generador estándar de Nest está pensado para MVC clásico, no para hexagonal.

---

5. Ejemplo completo, paso a paso

Vamos a construir un mini feature: crear y consultar productos. Vas a ver el flujo completo.

Paso 1 — La entidad de dominio (pura, sin Nest)

```ts
// src/products/domain/product.entity.ts

export class Product {
  private constructor(
    public readonly id: string,
    public readonly name: string,
    public readonly price: number,
  ) {}

  static create(params: { id: string; name: string; price: number }): Product {
    if (params.price < 0) {
      throw new Error('El precio no puede ser negativo');
    }
    if (!params.name.trim()) {
      throw new Error('El nombre es obligatorio');
    }
    return new Product(params.id, params.name, params.price);
  }
}
```

Fíjate: ni un solo `import` de `@nestjs/...`. Esto es TypeScript puro. Aquí viven las reglas de negocio ("el precio no puede ser negativo").

Paso 2 — El puerto de salida (la interfaz que el dominio necesita)

```ts
// src/products/domain/product-repository.port.ts

import { Product } from './product.entity';

export interface ProductRepository {
  save(product: Product): Promise<void>;
  findById(id: string): Promise<Product | null>;
}

// Token de inyección — en TS las interfaces no existen en tiempo de ejecución,
// así que necesitamos un "token" para que NestJS sepa qué inyectar.
export const PRODUCT_REPOSITORY = Symbol('PRODUCT_REPOSITORY');
```

Esto es lo más importante de todo el patrón en NestJS: las interfaces se borran al compilar TypeScript a JavaScript, así que no puedes decirle a Nest "inyéctame un `ProductRepository`" directamente. Por eso se usa un `Symbol` (o un string) como token, y luego se lo asocias a una implementación en el módulo (lo verás en el paso 5).

Paso 3 — El caso de uso (application layer)

```ts
// src/products/application/create-product.use-case.ts

import { Inject, Injectable } from '@nestjs/common';
import { Product } from '../domain/product.entity';
import { ProductRepository, PRODUCT_REPOSITORY } from '../domain/product-repository.port';
import { randomUUID } from 'crypto';

@Injectable()
export class CreateProductUseCase {
  constructor(
    @Inject(PRODUCT_REPOSITORY)
    private readonly productRepository: ProductRepository,
  ) {}

  async execute(input: { name: string; price: number }): Promise<Product> {
    const product = Product.create({
      id: randomUUID(),
      name: input.name,
      price: input.price,
    });

    await this.productRepository.save(product);

    return product;
  }
}
```

Nota: el `@Injectable()` y `@Inject()` sí son de Nest, porque este archivo vive en la capa "application" — que sí puede depender de Nest para aprovechar su inyección de dependencias. Lo único 100% "prohibido" de tocar Nest es la carpeta `domain/`.

Si quieres el dominio *absolutamente* aislado incluso de Nest en la capa de apli-

cación (para reutilizar la lógica en otro framework), puedes evitar los decoradores de Nest aquí también y hacer el cableado manual. Pero en el 95% de los proyectos NestJS, dejar `@Injectable()` en application está totalmente aceptado y es mucho más simple.

Paso 4 — El adaptador de salida (implementación real con TypeORM)

```
// src/products/infrastructure/persistence/product.orm-entity.ts

import { Entity, PrimaryColumn, Column } from 'typeorm';

@Entity('products')
export class ProductOrmEntity {
  @PrimaryColumn()
  id: string;

  @Column()
  name: string;

  @Column('decimal')
  price: number;
}
```

```
// src/products/infrastructure/persistence/product-typeorm.repository.ts

import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { ProductRepository } from '../../domain/product-repository.port';
import { Product } from '../../domain/product.entity';
import { ProductOrmEntity } from './product.orm-entity';

@Injectable()
export class ProductTypeOrmRepository implements ProductRepository {
  constructor(
    @InjectRepository(ProductOrmEntity)
    private readonly ormRepo: Repository<ProductOrmEntity>,
  ) {}

  async save(product: Product): Promise<void> {
    await this.ormRepo.save({
      id: product.id,
      name: product.name,
      price: product.price,
    });
  }

  async findById(id: string): Promise<Product | null> {
    const row = await this.ormRepo.findOneBy({ id });
    if (!row) return null;
    return Product.create({ id: row.id, name: row.name, price: row.price });
  }
}
```

Este archivo **sí** puede importar TypeORM, porque es infraestructura. Es un "traductor": convierte entre la entidad del dominio (`Product`) y la entidad de la base de datos (`ProductOrmEntity`). Nunca mezcles ambas.

Paso 5 — El adaptador de entrada (Controller)

```
// src/products/infrastructure/http/dto/create-product.dto.ts

import { IsString, IsNumber, Min } from 'class-validator';

export class CreateProductDto {
  @IsString()
  name: string;

  @IsNumber()
  @Min(0)
  price: number;
}
```

```
// src/products/infrastructure/http/product.controller.ts

import { Body, Controller, Post } from '@nestjs/common';
import { CreateProductUseCase } from '../../application/create-product.use-case';
import { CreateProductDto } from './dto/create-product.dto';

@Controller('products')
export class ProductController {
  constructor(private readonly createProductUseCase: CreateProductUseCase) {}

  @Post()
  async create(@Body() dto: CreateProductDto) {
    const product = await this.createProductUseCase.execute({
      name: dto.name,
      price: dto.price,
    });
    return { id: product.id, name: product.name, price: product.price };
  }
}
```

El controller es un adaptador de entrada: traduce HTTP → llamada al caso de uso. No tiene ninguna lógica de negocio, solo traduce.

Paso 6 — El módulo: aquí se conecta todo (el "cableado")

```
// src/products/products.module.ts

import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { ProductController } from './infrastructure/http/product.controller';
import { CreateProductUseCase } from './application/create-product.use-case';
import { PRODUCT_REPOSITORY } from './domain/product-repository.port';
import { ProductTypeOrmRepository } from './infrastructure/persistence/product-typeorm.reposito
import { ProductOrmEntity } from './infrastructure/persistence/product.orm-entity';
```

```
@Module({
  imports: [TypeOrmModule.forFeature([ProductOrmEntity])],
  controllers: [ProductController],
  providers: [
    CreateProductUseCase,
    {
      provide: PRODUCT_REPOSITORY,   // el "token" del puerto
      useClass: ProductTypeOrmRepository, // el adaptador real
    },
  ],
})
export class ProductsModule {}
```

Esta línea es la magia de todo el patrón:

```
{ provide: PRODUCT_REPOSITORY, useClass: ProductTypeOrmRepository }
```

Le dices a NestJS: "cuando alguien pida `PRODUCT_REPOSITORY`, dale una instancia de `ProductTypeOrmRepository`". Si mañana cambias a Mongo, solo cambias esa línea por `useClass: ProductMongoRepository` y **nada más en todo el proyecto se toca**.

---

6. ¿Qué se hace primero y por qué?

El orden importa: no es arbitrario. La lógica es **diseñar de adentro hacia afuera**.

1º — Dominio (entidad + reglas de negocio)

Empiezas por `Product` con sus reglas ("el precio no puede ser negativo", etc).

Por qué primero: es lo único que no depende de nada más. Si empezaras por el controller o la base de datos, estarías diseñando tu negocio *en función de* la tecnología ("¿qué campos tiene la tabla?") en vez de en función de las reglas reales ("¿qué necesita ser verdad para que un producto sea válido?"). Empezar aquí obliga a pensar primero en el problema, no en la herramienta.

2º — Puerto (la interfaz, ej. `ProductRepository`)

Justo después del dominio, defines qué necesita el dominio del mundo exterior.

Por qué segundo: el puerto lo escribes preguntándote "¿qué necesita mi caso de uso, no cómo lo voy a implementar?". Por eso el método se llama `save(product)` y no `insertIntoProductsTable(product)`. Si ya tienes en mente TypeORM antes de escribir esto, se te va a colar la tecnología en el nombre de los métodos.

3º — Caso de uso (application)

Con el dominio y el puerto listos, escribes la orquestación: `CreateProductUseCase`.

Por qué tercero: el caso de uso ya puede programarse completo aunque el repositorio real (TypeORM, Mongo, lo que sea) todavía **no exista**. Solo necesita la interfaz. En este punto ya puedes escribir y correr el test unitario con el `FakeProductRepository`, sin haber tocado infraestructura. Si logras testear tu negocio sin base de datos, el diseño va bien.

4º — Adaptador de salida (repositorio real, ej. Postgres/TypeORM)

Recién ahora decides la tecnología concreta y la implementas cumpliendo el contrato del puerto.

Por qué cuarto: es un detalle de implementación, literalmente lo último que "debería" importar. Si mañana cambias Postgres por Mongo, este es el único paso que se repite — todo lo anterior queda intacto.

5º — Adaptador de entrada (Controller)

Al final, conectas el "gatillo" que dispara el caso de uso desde afuera (HTTP).

Por qué al final: el controller es puro "pegamento" — recibe el request, valida formato, llama al caso de uso. No decide nada de negocio. Diseñarlo antes suele terminar en meter lógica de negocio ahí por comodidad (el error más común de principiante).

6º — Módulo (el cableado)

Lo último: conectas el token del puerto con la clase del adaptador en `providers`.

Por qué al final: es el resultado de todo lo anterior, no una decisión de diseño — solo le dices a Nest "cuando pidan esto, entrega aquello".

La regla mental que resume todo

Diseña de adentro hacia afuera, implementa de adentro hacia afuera, pero conecta de afuera hacia adentro.

Primero decides qué es tu negocio y qué necesita (dominio → puerto → caso de uso), y solo al final decides con qué tecnología lo vas a resolver (adaptadores) y cómo se dispara desde afuera (controller + módulo). Si te encuentras escribiendo primero el controller o la entidad de TypeORM, es señal de que la tecnología está definiendo tu negocio, y no al revés.

---

7. Cómo queda esto con PostgreSQL específicamente

El ejemplo de arriba ya usaba TypeORM, que es el ORM más común con Postgres en NestJS. Aquí el detalle completo de la parte de infraestructura, ya apuntando a Postgres real.

6.1 Instalar dependencias

```
npm install @nestjs/typeorm typeorm pg
```

- `pg` es el driver nativo de Node para Postgres. Sin esto, TypeORM no puede conectarse.

6.2 Variables de entorno (`.env`)

```
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=postgres
DB_PASSWORD=postgres
DB_NAME=mi_app
```

Instala también `@nestjs/config` (`npm install @nestjs/config`) para leerlas cómodamente.

6.3 Docker Compose para levantar Postgres en local

```yaml
# docker-compose.yml
version: '3.8'
services:
  postgres:
    image: postgres:16
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: mi_app
    ports:
      - '5432:5432'
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

```
docker compose up -d
```

6.4 Conectar TypeORM a Postgres (`app.module.ts`)

```typescript
// src/app.module.ts

import { Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';
import { ProductsModule } from './products/products.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    TypeOrmModule.forRootAsync({
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        type: 'postgres',
        host: config.get('DB_HOST'),
        port: +config.get('DB_PORT'),
        username: config.get('DB_USERNAME'),
        password: config.get('DB_PASSWORD'),
        database: config.get('DB_NAME'),
        entities: [__dirname + '/**/*.orm-entity{.ts,.js}'],
        synchronize: false, // en producción SIEMPRE false, usa migraciones
        // synchronize: true solo lo usas en desarrollo si quieres que
        // TypeORM cree las tablas automáticamente según tus entidades ORM.
      }),
    }),
    ProductsModule,
  ],
})
export class AppModule {}
```

Fíjate el patrón `entities: [__dirname + '/**/*.orm-entity{.ts,.js}']`: busca solo archivos que terminen en `.orm-entity.ts`, que es justo la convención que usamos para separar la entidad de infraestructura (`product.orm-entity.ts`) de la entidad de dominio (`product.entity.ts`). Esto evita que TypeORM intente "mapear" tu clase de dominio pura.

6.5 La entidad ORM de Postgres, con tipos correctos

```
// src/products/infrastructure/persistence/product.orm-entity.ts

import { Entity, PrimaryColumn, Column, CreateDateColumn } from 'typeorm';

@Entity('products')
export class ProductOrmEntity {
  @PrimaryColumn('uuid')
  id: string;

  @Column({ type: 'varchar', length: 150 })
  name: string;

  @Column({ type: 'numeric', precision: 10, scale: 2 })
  price: string; // ojo: Postgres devuelve 'numeric' como string en TypeORM

  @CreateDateColumn({ type: 'timestamptz' })
  createdAt: Date;
}
```

Detalle importante de Postgres: el tipo `numeric/decimal` lo devuelve TypeORM como `string`, no como `number`, para no perder precisión decimal (JavaScript no maneja bien decimales grandes). Por eso en el adaptador hay que convertir explícitamente:

```
// src/products/infrastructure/persistence/product-typeorm.repository.ts

import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { ProductRepository } from '../../domain/product-repository.port';
import { Product } from '../../domain/product.entity';
import { ProductOrmEntity } from './product.orm-entity';

@Injectable()
export class ProductTypeOrmRepository implements ProductRepository {
  constructor(
    @InjectRepository(ProductOrmEntity)
    private readonly ormRepo: Repository<ProductOrmEntity>,
  ) {}

  async save(product: Product): Promise<void> {
    await this.ormRepo.save({
      id: product.id,
      name: product.name,
      price: product.price.toFixed(2), // number -> string para 'numeric'
    });
  }
```

```
  async findById(id: string): Promise<Product | null> {
    const row = await this.ormRepo.findOneBy({ id });
    if (!row) return null;
    return Product.create({
      id: row.id,
      name: row.name,
      price: parseFloat(row.price), // string -> number para el dominio
    });
  }
}
```

Este mapeo (`toFixed` / `parseFloat`) es exactamente el tipo de "detalle sucio de la tecnología" que la arquitectura hexagonal existe para **contener**: vive únicamente aquí, en el adaptador. El dominio (`Product`) sigue trabajando con `number` sin enterarse jamás de que Postgres tiene esta particularidad.

6.6 Migraciones en vez de `synchronize: true`

En cualquier proyecto real con Postgres, no uses `synchronize: true` en producción (borra/altera tablas automáticamente y es peligroso). Se usan migraciones:

```
npm install -D typeorm-ts-node-commonjs

# generar una migración a partir de tus entidades ORM
npx typeorm-ts-node-commonjs migration:generate ./src/migrations/CreateProducts -d ./src/data-s

# aplicarla contra la base de datos
npx typeorm-ts-node-commonjs migration:run -d ./src/data-source.ts
```

Necesitas un `data-source.ts` separado (fuera de Nest) que TypeORM CLI pueda leer:

```
// src/data-source.ts
import { DataSource } from 'typeorm';

export default new DataSource({
  type: 'postgres',
  host: process.env.DB_HOST,
  port: +process.env.DB_PORT!,
  username: process.env.DB_USERNAME,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  entities: ['src/**/*.orm-entity.ts'],
  migrations: ['src/migrations/*.ts'],
});
```

6.7 Resumen: qué cambia y qué no cambia si usas Postgres

- **Lo que cambia:** solo `infrastructure/persistence/` (la entidad ORM y el repositorio) y la configuración de conexión en `app.module.ts`.
- **Lo que NO cambia nunca:** `domain/` (la entidad `Product` y el puerto `ProductRepository`), `application/` (el caso de uso), ni el controller.

Esa es la prueba real de que la arquitectura está bien hecha: Postgres es solo un detalle de implementación en una esquina del proyecto, intercambiable por Mongo, DynamoDB o lo que sea, sin tocar el resto.

8. El gran beneficio: testear sin NestJS

Como el caso de uso depende de una **interfaz**, no de TypeORM, puedes testearlo con un repositorio falso en memoria, sin levantar Nest ni base de datos:

```ts
// create-product.use-case.spec.ts

import { CreateProductUseCase } from './create-product.use-case';
import { ProductRepository } from '../domain/product-repository.port';
import { Product } from '../domain/product.entity';

class FakeProductRepository implements ProductRepository {
  private products: Product[] = [];

  async save(product: Product): Promise<void> {
    this.products.push(product);
  }

  async findById(id: string): Promise<Product | null> {
    return this.products.find((p) => p.id === id) ?? null;
  }
}

describe('CreateProductUseCase', () => {
  it('crea un producto válido', async () => {
    const repo = new FakeProductRepository();
    const useCase = new CreateProductUseCase(repo);

    const product = await useCase.execute({ name: 'Silla', price: 100 });

    expect(product.name).toBe('Silla');
    const saved = await repo.findById(product.id);
    expect(saved).not.toBeNull();
  });

  it('rechaza precio negativo', async () => {
    const repo = new FakeProductRepository();
    const useCase = new CreateProductUseCase(repo);

    await expect(
      useCase.execute({ name: 'Silla', price: -10 }),
    ).rejects.toThrow('El precio no puede ser negativo');
  });
});
```

Nada de `Test.createTestingModule`, nada de mockear TypeORM. Este test corre en milisegundos. **Esto es el verdadero premio de la arquitectura hexagonal**: tu negocio se puede probar aislado de todo.

---

9. Errores comunes de principiante (para que no caigas en ellos)

1. **Meter decoradores de TypeORM en la entidad de dominio.** Si tu `Product` tiene `@Entity()`, `@Column()`, ya no es dominio puro, es una entidad de infraestructura disfrazada. Usa dos clases separadas (dominio + entidad ORM) y un mapeo entre ellas, como en el ejemplo.
2. **Poner lógica de negocio en el Controller o en el Repository.** Esas capas solo "traducen". Si ves un `if` de reglas de negocio en un controller, muévelo al dominio o al caso de uso.
3. **Inyectar directamente `ProductTypeOrmRepository` en el caso de uso** en vez de la interfaz `ProductRepository`. Esto rompe todo el propósito: ya quedaste acoplado a TypeORM.
4. **Sobre-ingeniería en proyectos chicos.** Si tu API tiene 3 endpoints CRUD sin reglas de negocio reales, la arquitectura hexagonal puede ser más carga que beneficio. Este patrón brilla cuando hay lógica de negocio real que proteger y que necesitas testear mucho, o cuando esperas cambiar de infraestructura.
5. **Olvidar el token de inyección.** Como las interfaces de TypeScript desaparecen al compilar, si intentas hacer `@Inject(ProductRepository)` sin el `Symbol`, Nest no sabrá qué inyectar y te va a explotar en tiempo de ejecución.

---

10. Chuleta rápida (resumen mental)

- **domain/** → clases y reglas puras. Cero imports externos. Aquí van también los puertos (interfaces).
- **application/** → casos de uso. Orquestan el dominio. Pueden usar `@Injectable()` de Nest.
- **infrastructure/** → todo lo que toca el mundo real: controllers, repos de TypeORM/Prisma, clientes HTTP, colas, etc. Implementan los puertos.
- **El módulo (`*.module.ts`)** → el único lugar donde "conectas" el puerto con su adaptador real usando `{ provide: TOKEN, useClass: Implementación }`.
- Regla de dependencia: **todo apunta hacia adentro, hacia el dominio.** El dominio no conoce a nadie.

---

11. Siguiente paso natural: Domain-Driven Design (DDD)

La arquitectura hexagonal casi siempre viene acompañada de DDD cuando el proyecto crece: Value Objects, Aggregates, Domain Events, etc. No es obligatorio para empezar, pero si tu dominio se pone complejo, es el paso lógico después de dominar lo de esta guía.